



Cpp strings - gives details and codes on everything in the string library in c++

Computer science (University of Pretoria)



Scan to open on Studocu

C++ Strings – A Reference

```
#include <string>
using namespace std;
```

OR

```
#include <string>
using std::string;
```

```
string s1 = "hello";
string s2 = s1;           // s2 is "hello"
s1 = s1 + " there";      // s1 is "hello there"
```

```
if (s1 == s2)             // are they equal?
cin >> s1;               // Read chars from keyboard into s1 until white space is encountered.
getline(cin, s1);         // Read chars from keyboard into s1 until \n is encountered.
```

Prototype	Example	Description
<code>bool ← empty()</code>	<code>string a = "test"; if (a.empty()); // false</code>	Returns true if nothing is in the string, false otherwise.
<code>int ← length()</code>	<code>string a = "hello"; int len = a.length(); // 5</code>	Returns the number of chars in the string. Same as <code>size()</code> .
<code>char ← at(int pos)</code>	<code>string a = "hello"; char c = a.at(0); // same as c = a[0] a.at(0) = 'H'; // a is "Hello"</code>	Returns the char at position <i>pos</i> . Same functionality as using [<i>pos</i>] notation.
<code>char * ← c_str()</code>	<code>string a = "hello"; char *p = a.c_str(); // "hello"</code>	Returns a char array representing the string with a null terminator at the end.
<code>int ← compare(string s)</code>	<code>string a = "test"; if (a.compare("test") == 0) // same</code>	Compares the string to <i>s</i> , returns 0 if the string and <i>s</i> are equal, < 0 if the string is before <i>s</i> , and > 0 if after <i>s</i> . Just like < > ==.
<code>int ← compare(int start, int numChars, string s)</code>	<code>string a = "compare test"; if (a.compare(2, 3, "yes test") == 0) // compare "mpa" to "yes"</code>	Compares <i>numChars</i> chars starting at position <i>start</i> to string <i>s</i> . Note: g++ compiler requires string to be first arg, not last.
<code>erase(int start) erase(int start, int numChars)</code>	<code>string a = "erase this"; a.erase(0, 6); // Leaves "this" a.erase(2); // Leaves "th"</code>	Erases all chars to the right of the starting position <i>start</i> . Erases <i>numChars</i> number of chars beginning at position <i>start</i> .
<code>append(string s, int start, int numChars)</code>	<code>string a = "again"; a.append("state", 0, 2); // "against"</code>	Appends <i>numChars</i> chars from <i>s</i> to the string, beginning at slot <i>start</i> in <i>s</i> .
<code>resize(int size)</code>	<code>string a = "to"; a.resize(4); a[2] = "b"; a[3] = "e"; // a = "tobe"</code>	Resizes the string so it can hold additional characters.
<code>string ← substr(int start, int numChars)</code>	<code>string a = "a nice play"; string b = a.substr(2, 4); // "nice"</code>	Return a substring starting at slot <i>start</i> with <i>numChars</i> chars.
<code>int ← find(string s) int ← find(string s, int start) int ← rfind(string s) int ← rfind(string s, int start)</code>	<code>string a = "this is a test"; int pos = a.find("is"); // returns 2 pos = a.find("is", 3); // returns 5 pos = a.rfind("is"); // returns 5</code>	Returns the position of the first occurrence of <i>s</i> (starting at position 0 or <i>start</i>) or <code>string::npos</code> if the string is not found. Reverse find starts searching at back of string.
<code>int ← find_first_of(string s) int ← find_first_of(string s, int start) int ← find_last_of(string s) int ← find_last_of(string s, int start)</code>	<code>string a = "this is a test"; pos = a.find_first_of("iou"); // 2 pos = a.find_last_of("iou"); // 5</code>	Returns the first (or last) occurrence of any char in <i>s</i> or <code>string::npos</code> if none of the chars could be found. The parameter <i>start</i> can be used to start the search at that position.
<code>int ← find_first_not_of(string s) int ← find_last_not_of(string s)</code>	<code>string a = "this is a test"; pos = a.find_first_not_of("ihst"); // 4 pos = a.find_last_not_of("ihst"); // 11</code>	Returns first (or last) occurrence of any char not in <i>s</i> or <code>string::npos</code> if at least one of the chars in <i>s</i> could be found in every position of the string.
<code>replace(int start, int numChars, string s)</code>	<code>string a = "hello there"; a.replace(0, 5, "stay"); // "stay there"</code>	Replaces <i>numChars</i> chars starting at position <i>start</i> with <i>s</i> .
<code>insert(int start, string s)</code>	<code>a = "this is a test"; a.insert(8, "not "); // "this is not a test"</code>	Inserts <i>s</i> at position <i>start</i> in the string.
<code>swap(string s)</code>	<code>string a = "one", b = "two"; a.swap(b); // a = "two", b = "one"</code>	Swaps string <i>s</i> with the string.

* Note: There are many variations of the functions above which are not displayed here.

```
// Prints all items in a string that are separated by a common delimiter.
void parse(string parseString, string delimiter)
{
    string value;
    int startPos = 0, pos = parseString.find(delimiter);
    while (pos != string::npos)
    {
        value = parseString.substr(startPos, pos - startPos);
        cout << value << endl;
        startPos = pos + delimiter.length();
        pos = parseString.find(delimiter, startPos);
    }
    value = parseString.substr(startPos, parseString.length() - startPos);
    cout << value << endl;
}
```

```
// Example call:
parse("this::is::a::test", "::");
```